

IPL Win Predictor

System Design Document

Data Pipeline · Feature Engineering · ML Models · App Structure · Deployment

1. System Overview

The system has three parts that connect in a straight line — no complexity, no circular dependencies:

Part	Tool	What It Does
Part 1: Data Pipeline	Python script	Downloads Cricsheet JSON → parses → saves as clean CSV
Part 2: ML Pipeline	Jupyter Notebook	Loads CSV → engineers features → trains models → saves model.pkl
Part 3: Streamlit App	app.py (Python)	Loads model.pkl + CSV → takes user input → shows prediction + charts

Everything runs locally first. When done, Part 3 is pushed to Streamlit Cloud. That's the entire system — three Python files and one model file.

2. Data Design

2.1 Source

- Website: cricsheet.org/downloads — click 'IPL matches' to download free ZIP
- Format: one JSON file per match, each containing ball-by-ball data
- Size: roughly 900+ IPL matches from 2008 to 2024
- No login, no API key, no cost — just download and unzip

2.2 Raw JSON Structure (what one ball looks like)

```
{ 'over': 14, 'ball': 3, 'batter': 'MS Dhoni', 'bowler': 'Bumrah',  
  'runs': { 'batter': 1, 'extras': 0, 'total': 1 },  
  'wickets': [] }
```

You write a `parse_cricsheet.py` script that reads all JSON files and outputs one row per ball into a master CSV file.

2.3 Parsed CSV Columns (after your script runs)

Column	Type	Meaning
--------	------	---------

match_id	String	Unique ID per match — e.g. '2023_ipl_mi_csk'
batting_team	String	Team currently batting — e.g. 'Mumbai Indians'
bowling_team	String	Team currently bowling
city	String	Venue city — e.g. 'Mumbai', 'Chennai'
innings	Int (1 or 2)	1 = first innings, 2 = second innings (chase)
over	Float	Current over number — e.g. 14.3 means over 14 ball 3
runs_scored_so_far	Int	Total runs batting team has scored
wickets_fallen	Int	Number of wickets lost so far
target	Int	Runs needed to win (only relevant in 2nd innings)
result	Int (0 or 1)	1 = batting team won this match, 0 = lost — this is your label

3. Feature Engineering

This is the most important section. You do NOT train the model on raw columns. You create new meaningful columns. Each one has a real cricket reason — which is what you explain in interviews.

New Feature	Formula	Why You Created It (Interview Answer)
balls_left	$120 - (\text{over} \times 6)$	How many balls remain in the innings — captures urgency
runs_left	$\text{target} - \text{runs_scored_so_far}$	How many more runs needed to win — only 2nd innings
wickets_in_hand	$10 - \text{wickets_fallen}$	Remaining batting resources — higher = more comfortable
current_run_rate	$\text{runs_scored} / \text{overs_bowled}$	Scoring pace so far — CRR
required_run_rate	$\text{runs_left} / (\text{balls_left}/6)$	Pace needed to win — RRR. High RRR = pressure
crr_vs_rrr	$\text{current_run_rate} - \text{required_run_rate}$	Positive = batting team ahead, negative = behind

After feature engineering your final training dataframe has these columns as input (X) and 'result' as output (y): balls_left, runs_left, wickets_in_hand, current_run_rate, required_run_rate, crr_vs_rrr — plus one-hot encoded batting_team, bowling_team, city.

4. ML Model Design

4.1 Train/Test Split

- Filter to 2nd innings only — win/loss is meaningful only when chasing a target
- 80% train, 20% test — stratified by result so class balance is maintained
- No random_state leakage — use temporal split: train on 2008–2022, test on 2023–2024

4.2 Model 1 — Logistic Regression (Baseline)

- from sklearn.linear_model import LogisticRegression
- Why: fast, interpretable, gives probability output — good baseline
- Hyperparameters: C=1.0, max_iter=1000, solver='lbfgs'
- Expected accuracy: ~68–72%

4.3 Model 2 — XGBoost (Final Model)

- from xgboost import XGBClassifier
- Why better: handles non-linear relationships between CRR, RRR, wickets
- Hyperparameters: n_estimators=200, max_depth=6, learning_rate=0.05
- Expected accuracy: ~78–82%
- Save with: joblib.dump(model, 'ipl_model.pkl')

4.4 SHAP Explainability

- import shap — pip install shap
- explainer = shap.TreeExplainer(xgb_model)
- shap_values = explainer.shap_values(X_test)
- shap.summary_plot(shap_values, X_test) — generates feature importance bar chart
- This shows 'runs_left' and 'wickets_in_hand' are top predictors — explainable in plain English

4.5 Evaluation Metrics to Report

Metric	What It Means (Plain English)	Target
Accuracy	Correct predictions / total predictions	Target: > 75%
Precision	Of predicted wins, how many were actual wins	Shows false positive rate
Recall	Of actual wins, how many did model catch	Shows false negative rate
F1 Score	Harmonic mean of precision and recall	Overall balance metric
AUC-ROC	Area under the ROC curve	Target: > 0.82

5. Streamlit App Structure

The entire frontend is one Python file — app.py. Streamlit converts Python code into a web UI automatically. No HTML, no CSS, no JavaScript needed.

Page	Purpose	What User Sees
■ Predict	Main page	Dropdowns for teams + city. Sliders for score, overs, wickets. Shows win % gauge on submit.
■ Model Stats	Comparison page	Table comparing LR vs XGBoost accuracy, F1, AUC. ROC curve chart side by side.

■ Why This Prediction	SHAP page	SHAP bar chart for the current prediction. Plain-English explanation below the chart.
■ EDA Insights	Analysis page	Pre-computed charts: win % by venue, toss impact, team head-to-head. Loaded from saved PNG files.

5.1 File Structure (Your Entire Project)

```

ipl-win-predictor/
■■■ data/
    ■■■ raw/          # Cricsheet JSON files (gitignored – too large)
    ■■■ ipl_data.csv  # Your parsed clean CSV (committed)
■■■ notebooks/
    ■■■ analysis.ipynb # EDA + model training notebook
■■■ models/
    ■■■ ipl_model.pkl  # Saved XGBoost model
■■■ app.py             # Streamlit app – main file
■■■ parse_cricsheet.py # Data parsing script
■■■ requirements.txt   # pip install -r requirements.txt
■■■ README.md         # Screenshots + how to run

```

6. Deployment — 100% Free

Step	Action	Details
Step 1	Push to GitHub	git init → git add . → git commit → git push to a public GitHub repo
Step 2	Go to share.streamlit.io	Sign in with your GitHub account — free, no credit card
Step 3	Click New App	Select your repo, select app.py as the main file, click Deploy
Step 4	Get public URL	Streamlit gives you a URL like <code>pankaj-ipl-predictor.streamlit.app</code> — add this to your resume
Step 5	Add to README	Take a screenshot of the app, add it to README.md so GitHub preview looks good